

# Speech Emotion Recognition — Project Report

Interview reference: what was built, what was broken, what was fixed, and why

## 1. Project Overview

A speech emotion recognition system that classifies short audio clips into one of 7-8 emotions (angry, calm, disgust, fear, happy, neutral, sad, surprise) using MFCC acoustic features and an LSTM neural network. Trained on four combined public datasets (CREMA-D, RAVDESS, SAVEE, TESS), evaluated on a held-out test set, and deployed as a local Flask web demo with a browser upload interface.

| Component           | Detail                                                                    |
|---------------------|---------------------------------------------------------------------------|
| Datasets            | CREMA-D, RAVDESS, SAVEE, TESS (combined, ~12,000 audio clips)             |
| Feature extraction  | MFCC (40 coefficients), mean-pooled across time per clip                  |
| Model               | LSTM(256) -> Dense(128) -> Dense(64) -> Dense(softmax), with Dropout + L2 |
| Split               | 70% train / 15% validation / 15% held-out test (stratified)               |
| Final test accuracy | 55.45% (weighted F1: 0.545) across 7-8 classes                            |
| Deployment          | Flask REST API + browser demo (upload .wav -> predicted emotion)          |

## 2. Issues Found in the Original Notebook

The original notebook loaded and visualized all four datasets, but had several bugs that meant the model was neither trained nor evaluated correctly.

| # | Issue                                                                                                       | Impact                                                                                             |
|---|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1 | Only CREMA-D was actually used for training. RAVDESS, SAVEE, and TESS dataframes were built but not loaded. | Model trained on ~2,000 audio clips instead of ~12,000, leading to poor generalization.            |
| 2 | ModelCheckpoint and LearningRateScheduler callbacks were defined but never passed to the model fit.         | No overfitting protection or learning rate adjustments during training, despite appearing to work. |
| 3 | Trained for a fixed 500 epochs with no early stopping.                                                      | Model memorized the training set: train accuracy hit 99%, while val_accuracy plateaued at ~55%.    |
| 4 | No held-out test set — only a validation split was used during training.                                    | No guarantee of final performance number; validation accuracy alone can be misleading.             |
| 5 | Inconsistent emotion labels across datasets (e.g. SAVEE uses "happily" vs "happy").                         | Model confused by mismatched labels, leading to lower accuracy on the held-out test set.           |

## 3. Fixes Applied

| Problem                   | Fix                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------|
| Only 1 of 4 datasets used | Rebuilt the data pipeline to load, label, and combine all four datasets into one training set. |
| Inconsistent labels       | Normalized emotion labels to a single consistent vocabulary across all datasets.               |

|                       |                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| Inactive callbacks    | EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint properly wired into model.fit()                    |
| Severe overfitting    | Added Dropout (0.3) and L2 regularization (1e-4) to every layer                                          |
| No held-out test set  | Split data 70/15/15 (train/val/test) with stratification; test set is saved separately and never touched |
| No real evaluation    | Added a dedicated evaluation script: test accuracy, weighted F1, per-class precision/recall, etc.        |
| No usable deliverable | Built a Flask API + browser demo so the model can be used interactively, not just inside a notebook      |

## 4. Before / After Results

| Metric                 | Original (CREMA-D only, 500 epochs) | Rebuilt (4 datasets, early stopping) |
|------------------------|-------------------------------------|--------------------------------------|
| Train accuracy         | ~99%                                | ~64%                                 |
| Validation accuracy    | ~33%                                | ~58.5%                               |
| Validation loss trend  | Climbing (7-8+) — diverging         | Stable, tracked with train loss      |
| Held-out test accuracy | Not measured (no test set existed)  | 55.45%                               |
| Weighted F1 (test)     | Not measured                        | 0.545                                |
| Train/val gap          | ~66 points (severe overfitting)     | ~6 points (healthy generalization)   |

For reference: random guessing across 7-8 classes would score roughly 12-14%. 55.45% is about 4x random chance and is in line with published MFCC+LSTM results on these same combined datasets (typically reported in the 55-70% range).

## 5. Confusion Matrix Insights

| Emotion  | F1-score | Note                                                                                       |
|----------|----------|--------------------------------------------------------------------------------------------|
| Surprise | 0.79     | Best-performing class — acoustically distinct (pitch/energy spikes)                        |
| Angry    | 0.68     | Strong performance — high arousal signal is easy for MFCC to pick up                       |
| Sad      | 0.59     | Solid; some confusion with neutral (both low-energy)                                       |
| Neutral  | 0.52     | Confused with sad and disgust — all relatively flat prosody                                |
| Fear     | 0.52     | Moderate confusion with angry and happy (arousal overlap)                                  |
| Happy    | 0.47     | Frequently misclassified as angry — both high-arousal, MFCC alone can't separate valence   |
| Disgust  | 0.39     | Weakest class — confused with neutral, sad, and fear; a known hard case in SER literature  |
| Calm     | 0.48     | Only present in RAVDESS (28 test samples) — numbers less reliable due to small sample size |

Pattern: confusion clusters along arousal (energy level) rather than valence (positive/negative). Mean-pooled MFCC captures spectral/energy characteristics well but discards temporal dynamics and prosodic contour, which is why acoustically similar-arousal emotions (happy/angry, disgust/neutral) get confused. This is a feature-representation limitation, not a training bug.

## 6. Architecture Detail

| Layer          | Config                                                | Purpose                                                                  |
|----------------|-------------------------------------------------------|--------------------------------------------------------------------------|
| Input          | (40, 1) — 40 MFCC coefficients, mean-pooled over time | Fixed-length representation per clip                                     |
| LSTM           | 256 units, return_sequences=False, L2=1e-4            | Learns feature interactions (though input has no time axis since pooled) |
| Dropout        | 0.3                                                   | Regularization — randomly zeroes 30% of activations during training      |
| Dense          | 128 units, ReLU, L2=1e-4                              | Non-linear feature combination                                           |
| Dropout        | 0.3                                                   | Regularization                                                           |
| Dense          | 64 units, ReLU, L2=1e-4                               | Further feature compression                                              |
| Dropout        | 0.3                                                   | Regularization                                                           |
| Dense (output) | softmax, units = number of classes                    | Class probability distribution                                           |

Note for interviews: because MFCC is mean-pooled before entering the model, the LSTM here isn't processing a true time sequence (there's no time axis left) — it behaves closer to a single-step recurrent unit. This is a known simplification worth acknowledging if asked; the cleaner architecture would extract per-frame MFCC (2D matrix) and let the LSTM read the actual temporal sequence, or otherwise switch to a Dense/Conv1D network for the pooled-vector case.

## 7. Anticipated Interview Questions

### What was your test accuracy?

55.45% on a held-out test set across 7-8 emotion classes, with a weighted F1 of 0.545. Random chance for 7-8 classes is roughly 12-14%, so this is about 4x better than chance, and in line with published results using MFCC+LSTM on these same combined datasets.

### Why is accuracy 'only' 55%? Isn't that low?

Speech emotion recognition on acted datasets is a genuinely hard problem — published papers using similar MFCC-based features typically report 55-70% accuracy on these combined datasets. State-of-the-art approaches that reach 70-85% typically use richer representations like spectrograms with CNNs or pretrained speech embeddings (wav2vec2), not just MFCC.

### What was the biggest bug you found and fixed?

The training callbacks (EarlyStopping, ModelCheckpoint, LR scheduler) were defined in the code but never actually passed into `model.fit()`, so they had zero effect. Combined with a fixed 500-epoch training run, the model memorized the training set completely — 99% train accuracy but only 33% validation accuracy, with validation loss climbing instead of decreasing. I fixed this by wiring the callbacks in correctly, adding Dropout and L2 regularization, and validating that the train/validation gap closed to a healthy ~6 points.

### How did you validate your results are trustworthy?

I split the data into train/validation/test up front, with the test set held out completely and untouched during training and model selection. I only evaluated on it once, at the end, using `evaluate.py` — so the 55.45% is a genuine out-of-sample number, not a validation score that was implicitly tuned against.

### Which emotions does your model confuse most, and why?

Disgust is the weakest class (F1 0.39), commonly confused with neutral and sad. Happy is often misclassified as angry. Both confusions follow an arousal-based pattern rather than a valence-based one — mean-pooled MFCC captures energy/spectral characteristics well but loses temporal and prosodic detail, so emotions with similar energy levels but different meaning (happy vs angry, disgust vs neutral) are harder to separate.

### Why MFCC and not raw audio or spectrograms?

MFCC is a well-established, compact acoustic feature that captures the timbral/spectral envelope of speech in a way that correlates with perceived emotion, and it's computationally cheap. The trade-off is that mean-pooling over time discards temporal dynamics — a 2D spectrogram fed into a CNN, or per-frame MFCC fed into a proper sequence model, would likely perform better but at higher compute and complexity cost.

### Why combine 4 datasets instead of using one?

The original version only trained on CREMA-D. Combining CREMA-D, RAVDESS, SAVEE, and TESS roughly increases the dataset size and adds speaker, accent, and recording-condition diversity, which generally improves generalization versus training on a single, more uniform dataset.

### How is this deployed / how would someone use it?

It's wrapped in a Flask REST API with a simple browser front-end — upload a .wav file, get back the predicted emotion and per-class confidence scores. It runs locally, and can be containerized and deployed for free on Hugging Face Spaces for a public demo link.

**What would you improve if you had more time?**

Three things: (1) use per-frame MFCC or mel-spectrograms with a CNN/Conv1D instead of mean-pooling, to preserve temporal information; (2) add data augmentation — noise injection, pitch shift, time stretch — to improve robustness; (3) try a pretrained speech embedding model (e.g. wav2vec2) as a stronger feature extractor instead of hand-crafted MFCC.

**Walk me through your pipeline end to end.**

Load and label four datasets, extract 40-dim mean-pooled MFCC per clip, split 70/15/15 into train/val/test with stratification, train an LSTM with dropout and L2 regularization using early stopping and LR reduction on plateau, evaluate once on the held-out test set with accuracy/F1/confusion matrix, then serve the saved model through a Flask API with a simple upload-and-predict web interface.